

# Poster (eng. records)

---

- Post heter `struct` i C
  - slå ihop flera värden till ett
  - dessa kan då betraktas som *ett* värde
  - *eller* som sina ingående delar
  - sätt bra *namn* på delarna
- Enkelt att förstå och mycket bra för god programstruktur

# struct

---

```
struct person      /* tala om hur alla personer ser ut */
{                  /* läggs oftast i en .h-fil          */
    char namn[40];
    int id;
    float vikt;
};                  /* semikolon!! */

void skrivperson( struct person p);    /* prototyp */

int main()
{
    struct person p1, p2;
    scanf("%s", p1.namn);
    p1.id = 4711;    p1.vikt = 68.78;
    p2 = p1;
    skrivperson(p1); skrivperson(p2);
}
```

# struct som parameter

---

```
void skrivperson(struct person p)
{
    printf("Namn: %s\n", p.namn);
    printf("Id: %d\n", p.id);
    printf("Namn: %f\n", p.vikt);
}
```

skrivperson får en *kopia* av p1 och p2  
Det känns lite onödigt, i synnerhet om  
struct person vore mycket större

# Pekare till struct

---

- Det går bra att deklarera pekare till struct

```
struct person p3;  
struct person *ppek;  
void skrivperson_p( struct person *pp);  
ppek = &p3;  
scanf("%s %d", p3.namn, &p3.id);  
(*ppek).vikt = 78.5;  
ppek->vikt = 78.5;      /* enklare skrivsätt för samma sak */  
skrivperson_p(ppek);  
skrivperson_p(&p3);
```

# struct-pekare som parameter

---

```
void skrivperson_p(struct person *pp)
{
    printf("Namn: %s\n", pp->namn);
    printf("Id: %d\n", pp->id);
    printf("Namn: %f\n", pp->vikt);
}
```

Nu behöver vi bara kopiera pekare vid anropet!

# Mera om struct

---

- Man kan lägga in pekare i struct-er
  - bygga länkade strukturer
  - träd, tabeller, nät etc
- Man kan allokera struct-er med malloc
  - dynamiskt bygga upp strukturer som är mycket flexibla och effektiva

# Typdefinitioner

---

## ► Man kan sätta nya namn på typer

– bra för läsbarhet och ändringsbarhet

```
typedef float yta;    /* yta betyder nu float */
typedef float volym; /* volym betyder nu float */

volym v1, burk;
yta y1;
void fyll(volym vv);

y1 = 4.5;
v1 = y1;    /* OK(!), tyvärr blev det ingen ny typ
              bara ett nytt namn på en gammal typ */
```

# Typdefinitioner av struct

---

- Man kan sätta nya namn på struct-typer
  - om man vill slippa skriva struct

```
typedef
struct person
{
    char namn[40];
    int id;
    float vikt;
} Person;                /* vi *kunde* ha använt
                           person med litet p */
```

```
Person p1, p2;
```



# Preprocessorn och C-bibliotek

---

- Makro, include-filer och villkorlig kompilering
- Standardbibliotek i ANSI-C
  - in/ut-matning
  - textsträngar
  - datum och tid
- Skapa egna bibliotek

# Preprocessorn

---

- En ”dum” textutbytare som körs innan kompilatorn får C-koden
  - makron byts ut
  - include-filers innehåll tas med
  - m.m.
- Gör inget komplicerat!
  - lätt att göra svårupptäckta fel

# Makron

---

- definieras med `#define`
  - byts ut i resten av texten

```
#define SIZE 100
```

```
int iflt[SIZE]; int i;  
printf("Fältets SIZE = %d", SIZE);  
for (i=0; i<SIZE; i++)  
    iflt[i] = 0;
```

# Makro med parametrar

---

- Kan låtsas vara funktion
  - gör inget svårt, stor risk för fel
  - preprocessorn kan inte C

```
#define printint(x)    printf( "%d", x )
```

```
int i = 6;
```

```
printint(i);
```

```
printint(i + 78);
```

# #include

---

## ► Byt ut raden mot filinnehållet

```
#include <stdio.h>
```

```
#include <math.h>          /* standard include-filer */
```

```
#include "buffert.h"       /* egen include fil */
```

# Villkorlig kompilering

---

## ► Tag med kod villkorligt

```
#ifdef UNIX
#define KVALITET 89
    fork();
#endif
```

```
#ifdef W95
#define KVALITET 37
    avbrytgenast();
#endif
```

Finns i några andra former också

# ANSI C bibliotek

---

- Standardiserades ihop med språkstandarden
  - mycket ärvt från UNIX
  - inte alltid den bästa kvaliteten
- Till varje del i biblioteken hör en include-fil

```
#include <stdio.h>  /* prototyper för i/o  
                    bibliotekets funktioner */  
  
/* .... */  
    printf( "%d", 78);  
    putchar( 'A' );
```

# Standard i/o

---

- ▶ Vi har redan använt en del
  - `printf`, `scanf`, `getchar`, `putchar`
- ▶ För att skriva på filer finns en speciell typ `FILE`

```
#include <stdio.h>

FILE *ut;

ut = fopen("resultat", "w"); /* öppna för skrivning */
                               /* "a" för tillägg i fil */

fprintf(ut, "Hej, hopp hamnar i filen!\n");
fprintf(stdout, "Detta hamnar nog på skärmen\n");
fprintf(stderr, "Detta hamnar säkert på skärmen\n");
```



# Standard i/o

---

## ► Motsvarande för läsning

```
#include <stdio.h>

FILE *in;

char buf[500]

in = fopen("resultat", "r");    /* öppna för läsning */
fscanf(in, "%s", buf);         /* läs första ordet */
printf("%s", buf);             /* skriv på skärmen */
```

# Strängar

---

- Många hjälprutiner för strängbearbetning
  - nästan alla kräver noggrannhet av *användaren*

```
#include <string.h>
```

```
char *namn = "Ulf";      char *kopia;      char buf[100];
```

```
strcpy(kopia, namn);     /* kopierar till odefinierad plats! */
```

```
strcpy(buf, namn);       /* OK, "Ulf" får plats i buf          */
```

```
kopia = malloc(strlen(namn) + 1);
```

```
strcpy(kopia, namn);     /* lagom stor för "Ulf"           */
```

# Datum och tid

---

```
#include <time.h>

char *dagnamn[7] = {"söndag", "måndag", "tisdag", "onsdag",
                    "torsdag", "fredag", "lördag"};

time_t nuticks;

struct tm *nu;    /* time_t och struct tm från time.h */

time(&nuticks);    /* i UNIX: sekunder sedan 1970-01-01 */
nu = localtime(&nuticks);

printf("Klockan är %02d:%02d", nu->tm_hour, nu->tm_min);
printf("Det är %s\n", dagnamn[nu->tm_wday]);
```

# Diverse

---

## ► Det finns allt möjligt

```
#include <stdlib.h>

int i;

srand(time(0));    /* skaka om slumpen */

i = rand() % 100; /* ett slumptal 0-99 */


exit(1);           /* avbryt programkörningen direkt */


system("ls -l");   /* kör ett shell-kommando */
```

# Egna bibliotek

---

- ▶ Gör en snygg design
  - välbesigade moduler med väl valda funktioner, parametrar och dataobjekt
  - definitioner i `.c`-filer
  - deklarationer (funktionsprototyper, typedef, struct, enum etc) i `.h`-filer
- ▶ Kompilera `.c`-filer till `.o`-filer
  - man kan samla många `.o`-filer i ett bibliotek med UNIX-kommandot `ar`
  - förpacka och sälj bibliotek och `.h`-filer